LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

“Analisis Kompleksitas Algoritma”

DISUSUN OLEH:

AULIA MURSYIDA

NIM : 2025573010006

KELAS : TI 1D

JURUSAN : Teknologi Informasi Dan Komputer

PRODI : Teknik Informatika

MATKUL : Praktikum Algoritma Dan Struktur Data

DOSEN PENGAJAR : Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026/2027

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan karunia-Nya, sehingga saya dapat menyelesaikan laporan ini dengan baik. Laporan ini dibuat untuk memenuhi tugas praktikum mata kuliah Praktikum Algoritma Dan Struktur Data.

Adapun laporan ini dibuat untuk memenuhi tugas praktikum serta untuk menambah pemahaman mengenai efisiensi algoritma, notasi Big O, space complexity, dan pola kompleksitas. Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan. Oleh karena itu, saya sangat menerima kritik dan saran dari pembaca demi perbaikan laporan di masa yang akan datang.

Buket Rata, 29 April 2026

Aulia Mursyida

NIM. 2025573010006

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I 1
PENDAHULUAN 1
1.1 LATAR BELAKANG 1
1.2 TUJUAN 1
BAB II 2
LANDASAN TEORI 2
2.1 Efisiensi Algoritma 2
2.2 Notasi Biog O 2
2.3 Space Complexity 3
2.4 Mengenali Pola Kompleksitas 3
BAB III 5
PEMBAHASAN 5
3.1 Mempersiapkan Atau Membuat Folder Projek. 5
3.2 Membuat File 01-function-dasar.js 6
3.3 Membuat File 02-kelas-big-o.js 7
3.4 Membuat Latihan di dalam file baru Bernama Latihan-1.js 8
3.5 Membuat File 03-space-complexity.js 10
3.7 Membuat Latihan didalam file 03-space-complexity.js 11
3.8 Membuat File 04-refactor.js 13
3.11 Membuat Folder dan File Tugas 14
3.12 Membuat Tugas 1 14
3.13 Membuat Tugas 2 16
3.14 Mengumpulkan Tugas di Repository 18
BAB IV 20
ANALISIS 20
4.1 Analisis Efisiensi Algoritma 20
4.2 Analisi notasi big O 20
4.3 Analisis Space Complexity 20
4.4 Analisis Mengenali Pola Kompleksitas 21

4.5 Stack dan Queue 21
BAB V 22
PENUTUP 22
5.1 Kesimpulan 22

BAB I
PENDAHULUAN

1.1 LATAR BELAKANG

Dalam dunia pemrograman, algoritma menjadi dasar dalam menyelesaikan suatu permasalahan. Namun, tidak semua algoritma memiliki tingkat efisiensi yang sama. Oleh karena itu, penting untuk memahami bagaimana mengukur dan menganalisis efisiensi algoritma. Dengan adanya konsep seperti Big O dan space complexity, seorang programmer dapat memilih algoritma yang lebih optimal sehingga program dapat berjalan lebih cepat dan efisien, terutama dalam pengolahan data yang besar. Selain itu, pemahaman ini juga membantu dalam mengurangi penggunaan sumber daya komputer serta meningkatkan performa sistem secara keseluruhan.

1.2 TUJUAN

Tujuan dari praktikum ini adalah:

1. Memahami pentingnya efisiensi algoritma.
2. Mengetahui konsep Big O Notation.
3. Menganalisis time complexity dan space complexity.
4. Mampu menegenali pola kompleksitas algoritma.

BAB II
LANDASAN TEORI

2.1 Efisiensi Algoritma

Efisiensi algoritma adalah cara untuk menilai seberapa baik suatu algoritma dalam menyelesaikan masalah dengan mempertimbangkan waktu eksekusi dan penggunaan memori. Algoritma yang efisien mampu bekerja lebih cepat dan menggunakan sumber daya lebih sedikit, terutama saat jumlah data semakin besar. Untuk mengukur efisiensi, digunakan konsep Big O, yang menggambarkan pertumbuhan waktu atau memori terhadap ukuran input (n). Secara umum, efisiensi dibagi menjadi dua, yaitu time complexity (waktu) dan space complexity (memori). Semakin kecil nilai kompleksitas suatu algoritma, maka semakin baik dan efisien algoritma tersebut.

2.2 Notasi Biog O

Big O adalah cara untuk mengukur seberapa cepat atau lambat suatu algoritma berjalan saat jumlah data bertambah. Dalam notasi Big O, terdapat beberapa jenis kompleksitas yang menunjukkan tingkat efisiensi algoritma berdasarkan pertumbuhan input (n), yaitu:

- O(1) (Konstan)
  Waktu eksekusi tidak dipengaruhi oleh jumlah data. Selalu cepat.
- O(log n) (Logaritmik)
  Waktu bertambah secara lambat karena data dibagi menjadi beberapa bagian. Contohnya binary search.
- O(n) (Linear)
  Waktu eksekusi sebanding dengan jumlah data. Semakin banyak data, semakin lama waktu yang dibutuhkan.

- O(n log n)
  Gabungan linear dan logaritmik, biasanya digunakan pada algoritma sorting yang efisien.
- $O(n^2)$ (Kuadratik)
  Biasanya terjadi pada nested loop. Waktu eksekusi meningkat cukup cepat saat data bertambah.
- $O(2^n)$ (Eksponensial)
  Kompleksitas sangat tinggi dan tidak efisien untuk data besar.

2.3 Space Complexity

Space complexity adalah ukuran yang digunakan untuk mengetahui jumlah memori tambahan (auxiliary space) yang dibutuhkan oleh suatu algoritma seiring bertambahnya ukuran input. Perhitungan ini tidak termasuk data input utama, melainkan hanya memori tambahan yang digunakan selama proses eksekusi. Dalam space complexity, penggunaan memori biasanya dinyatakan dalam notasi Big O, seperti O(1) untuk penggunaan memori tetap dan O(n) untuk penggunaan memori yang bertambah sesuai jumlah data. Terdapat trade-off antara waktu dan memori, di mana suatu algoritma bisa lebih cepat dengan menggunakan lebih banyak memori, atau sebaliknya.

2.4 Mengenali Pola Kompleksitas

Mengenali pola kompleksitas adalah cara untuk memahami tingkat efisiensi algoritma dengan melihat struktur kode yang digunakan, tanpa harus menjalankan program. Dengan mengenali pola ini, kita dapat memperkirakan kompleksitas waktu suatu algoritma secara cepat.

Beberapa pola kompleksitas yang umum adalah:

1. Akses langsung ke array atau object : O(1)
   Contoh: arr[0], obj.key, atau Map.get().
2. Satu loop melewati n elemen : O(n)
   Digunakan pada perulangan seperti for atau forEach..

3. Loop yang membagi data (i*=2 atau i/=2) : O(log n)

   Biasanya ditemukan pada algoritma seperti binary search.

4. Dua loop berurutan (tidak bersarang) → O(n)

   Kompleksitas tetap linear karena konstanta diabaikan (2n → O(n)).

5. Nested loop (dua loop bersarang) → $O(n^2)$

   Contohnya pada algoritma bubble sort atau pencarian pasangan.

6. Tiga loop bersarang → $O(n^3)$

   Jarang digunakan, biasanya pada operasi matriks.

7. Rekursi bercabang dua → $O(2^n)$

   Contohnya pada Fibonacci rekursif tanpa optimasi.

8. Rekursi dengan pembagian dan penggabungan → O(n log n)

   Contohnya pada merge sort dan quick sort.

BAB III

PEMBAHASAN

3.1 Mempersiapkan Atau Membuat Folder Projek.

1. Membuka Git Bash.

   Buka aplikasi git bash di tombol start atau windows. Kemudian navigasikan ke root folder repository.

```
MINGW64:/d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-5

ACER@LAPTOP-ENDOAFFH MINGW64 ~ (master)
$ cd /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma
```

2. Membuat folder Projek.

   Ketik mkdir praktikum-5 untuk memasukkan atau membuat folder baru bernama praktikum-5 dan untuk menyimpan semua file. Selanjutnya, ketik cd praktikum-5 untuk menjalankan projek.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ mkdir praktikum-5

ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ cd praktikum-5
```

3. Inisialisasi project.

   Ketik npm init -y untuk menginisialisasi.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ npm init -y
Wrote to D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3.2 Membuat File 01-function-dasar.js

1. Di dalam folder praktikum-5, buat file baru bernama 01-intro-kompleksitas.js. Ketik perintah touch 01-intro-kompleksitas.js

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ touch 01-intro-kompleksitas.js
```

2. Buka file 01-intro-kompleksitas.js di aplikasi VS Code, lalu ketikkan kode sesuai di modul praktikum yang telah dikirimkan oleh dosen.

```
JS 01-intro-kompleksitas.js ×
JS 01-intro-kompleksitas.js > ...
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}

const N = 100_000; // ukuran data: 100 ribu

function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++) total += i;
    return total;
}

function jumlahkanRumus(n) {
    return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
    return pasangan;
}
const data = Array.from({length: 5000}, (_, i) => i);

console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('O(n^2) cariPasangan ', () => cariPasangan(data));

console.log('\nHasil sama? ', jumlahkanLinear(100) === jumlahkanRumus(100));
```

3. Simpan File 01-intro-kompleksitas.js tersebut, kemudian jalankan file tersebut dari terminal dengan cara ketikkan node 01-intro-kompleksitas.js.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 1 ms
O(n^2) cariPasangan : 15 ms

Hasil sama?  true
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.3 Membuat File 02-kelas-big-o.js

1. Membuat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5 dengan cara ketik perintah touch 02-kelas-big-o.js

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatada
nalgoritma/praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

2. Buka file 02-kelas-big-o.js di aplikasi VS Code, lalu ketikkan kode sesuai di modul praktikum yang telah dikirimkan oleh dosen.

```
JS 02-kelas-big-o.js ×   JS 02-destructuring-spread.js
JS 02-kelas-big-o.js > ...
function ambilPertama(arr) { return arr[0]; }
function tambahkanItem(arr, item) { arr.push(item); }
function isGenap(n) { return n % 2 === 0; }

function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
    return a;
}

function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2);
}

console.log('=== O(1) - selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50]));
console.log(isGenap(42));

console.log('\n=== O(log n) - Binary Search ===');
const sorted = Array.from({length:1_000_000},(_,i)=>i);
binarySearch(sorted, 731_452);

console.log('\n=== O(n) - Linear Search ===');
console.log('Max dari 1000 elemen:',
cariMax(Array.from({length:1000}, ()=>Math.random()*1000|0)));

console.log('\n=== O(n^2) - Bubble sort (n kecil!) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log('\n=== O(2^n) - Fibonacci Rekursif (n kecil!) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');

console.log('\nWaktu fib(35) O(2^n):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');

console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return n;
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

3. Simpan file 02-kelas-big-o.js, kemudian jalankan file tersebut dari terminal dengan cara ketik node 02-kelas-big-o.js.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\02-kelas-big-o.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 997

=== O(n^2) - Bubble sort (n kecil!) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) - Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
105 ms
Waktu fib(35) O(n) memoization:
0 ms
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.4 Membuat Latihan di dalam file baru Bernama Latihan-1.js

1. Pada latihan ini dianalisis beberapa fungsi untuk menentukan kompleksitas Big O. Fungsi A memiliki O(1), fungsi B O(n^2), fungsi C O(log n), dan fungsi D O(n^3). Setiap fungsi diuji dengan n = 1000 menggunakan Date.now() melalui fungsi hitungKompleksitas. Hasilnya menunjukkan bahwa semakin besar kompleksitas, semakin lama waktu eksekusi.

02-kelas-big-o.js | tugas-1.js | tugas-2.js | latihan-1.js

latihan-1.js > ...

```javascript
//fungsi A
function fnA(n){
    return n * 2;
}
// big o, karena operasi nya hanya 1 kali saja, dan tidak berganti kepada n

//fungsi b
function fnB(n){
    for (let i = 0; i < n; i++){
        for(let j = 0; j < n; j++){
            console.log(i,j);
        }
    }
}

//big o(n^2), karena perulangan bersarang, n x n

// fungsi c
function fnC(n){
    for(let i = 1; i < n; i *= 2){
        console.log(i);
    }
}

//big o(log n), karena i dikali kan 2 setiap iterasi nya

//fungsi d
function fnD(arr){
    arr.forEach(x => {
        arr.forEach(y =>{
            arr.forEach(z => {
                console.log(x,y,z);
            });
        });
    });
}

//big O(n^3), 3 loop bersarang, n x n x n

function hitungKompleksitas(n, fn){
    const start = Date.now();

    fn(n);

    const end = Date.now();
    console.log(`waktu eksekusi: ${end - start} ms`);
}

function fnB(n){
    for (let i = 0; i < n; i++){
        for (let j = 0; j < n; j++){

        }
    }
}

function fnc(n){
    for(let i = 1; i < n; i *= 2){}
}

function fnD(arr){
    arr.forEach(x => {
        arr.forEach(y =>{
            arr.forEach(z => {

            });
        });
    });
}

console.log('fungsi A');
hitungKompleksitas(1000,fnA);

console.log('fungsi B');
hitungKompleksitas(1000,fnB)

console.log('fungsi C');
hitungKompleksitas(1000,fnC)

console.log('fungsi D');
hitungKompleksitas(1000, (n) => {
    const arr = new Array(n).fill(0);
    fnD(arr);
});
```

2. Simpan dan jalankan di terminal.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\latihan-1.js
fungsi A
waktu eksekusi: 0 ms
fungsi B
waktu eksekusi: 1 ms
fungsi C
1
2
4
8
16
32
64
128
256
512
waktu eksekusi: 2 ms
fungsi D
waktu eksekusi: 403 ms
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.5 Membuat File 03-space-complexity.js

1. Membuat file baru bernama 03-space-complexity.js di dalam folder praktikum-5 dengan cara ketik perintah touch 03-space-complexity.js

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ touch 03-space-complexity.js
```

2. Buka file 03-space-complexity.js di aplikasi VS Code, lalu ketikkan kode sesuai di modul yang telah dikirimkan oleh dosen.

```
JS 03-space-complexity.js ×
JS 03-space-complexity.js > ...
function jumlahArray(arr) {
    let total = 0;
    for (const x of arr) total += x;
    return total;
}

function duplikasiArray(arr) {
    const baru = [];
    for (const x of arr) baru.push(x * 2);
    return baru;
}

function faktorialRekursi(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursi(n - 1);
}

function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i;
    return hasil;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log('Jumlah array :', jumlahArray(arr));
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursi(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));

function hitungUnik(arr) {
    const seen = new Set();
    for (const x of arr) seen.add(x);
    return seen.size;
}

const dataAcak = [1, 2, 3, 4, 5, 3, 6, 4, 7];
console.log('Elemen unik:', hitungUnik(dataAcak));
```

3. Simpan File 03-space-complexity.js, kemudian jalankan file tersebut dari terminal dengan cara ketikkan node 03-space-complexity.js

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.7 Membuat Latihan didalam file 03-space-complexity.js

1. Pada latihan ini dibuat dua fungsi, yaitu cariPasanganLambat dan cariPasanganCepat. Fungsi lambat menggunakan nested loop dengan kompleksitas $O(n^2)$ dan O(1) space, sedangkan fungsi cepat menggunakan Set dengan kompleksitas O(n) dan O(n) space. Pengujian dengan data [2, 7, 11, 15] dan target 9 menghasilkan [2, 7] pada kedua fungsi. Saat diuji dengan data besar, fungsi cepat terbukti lebih efisien dibandingkan fungsi lambat. Trade-off yang terjadi adalah fungsi cepat membutuhkan memori tambahan, sedangkan fungsi lambat tidak, tetapi memiliki waktu eksekusi lebih lama.

```javascript

console.log('\nLATIHAN 3');
class BankAccount {
    constructor (owner, balance = 0) {
        this.owner = owner;
        this.balance = balance;
    }
    deposit(amount) {
        this.balance += amount;
    }
    withdraw(amount) {
        if (amount <= this.balance) {
            this.balance -= amountt;
        }
    }
}

function cariPasanganLambat(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }
        }
    }
    return null;
}

function cariPasanganCepat(arr, target) {
    const seen = new Set();

    for (let num of arr) {
        let complement = target - num;

        if (seen.has(complement)) {
            return [complement, num];
        }
        seen.add(num);
    }

    return null;
}

const arr1 = [2, 7, 11, 15];
const target1 = 9;

console.log('Lambat:', cariPasanganLambat(arr1, target1));
console.log('Cepat:', cariPasanganCepat(arr1, target1));

const arr2 = [];
for (let i = 0; i < 50000; i++) {
    arr2.push(Math.floor(Math.random() * 100000));
}

const target2 = -999999;

console.time('Lambat 50k');
cariPasanganLambat(arr2, target2);
console.timeEnd('Lambat 50k');

console.time('Cepat 50k');
cariPasanganCepat(arr2, target2);
console.timeEnd('Cepat 50k');

```

2. Simpan File, kemudian jalankan file tersebut dari terminal dengan cara ketik node 03-space-complexity.js

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

LATIHAN 3
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
Lambat 50k: 1.317s
Cepat 50k: 5.459ms
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.8 Membuat File 04-refactor.js

1. Membuat file baru bernamaa 04-refactor.js di dalam folder praktikum-5 dengan cara ketik perintah touch 04-refactor.js

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ touch 04-refactor.js
```

2. Buka file 04-refactor.js di aplikasi VS Code, lalu ketikkan kode sesuai di modul praktikum yang telah dikirimkan oleh dosen.

```
04-inheritance.js
04-inheritance.js > ...
class Kendaraan {
    constructor(merk, model, tahun) {
        this.merk = merk;
        this.model = model;
        this.tahun = tahun;
        this.kecepatanSaatIni = 0;
    }

    akselerasi(tambahan) {
        this.kecepatanSaatIni += tambahan;
        console.log(`${this.merk} ${this.model}: kecepatan ${this.kecepatanSaatIni} km/h`);
    }

    rem() {
        this.kecepatanSaatIni = 0;
        console.log(`${this.merk} ${this.model}: berhenti. `);
    }

    info() {
        return `${this.tahun} ${this.merk} ${this.model}`;
    }
}

class Mobil extends Kendaraan {
    constructor(merk, model, tahun, jumlahPintu) {
        super(merk, model, tahun);
        this.jumlahPintu = jumlahPintu;
    }

    bunyikanKlakson() {
        console.log(`${this.merk}: Beep beep!`);
    }

    info() {
        const infoParent = super.info();
        return `${infoParent} (${this.jumlahPintu} pintu)`;
    }
}

class Motor extends Kendaraan {
    constructor(merk, model, tahun, jenisMotor) {
        super(merk, model, tahun);
        this.jenisMotor = jenisMotor;
    }

    wheelie() {
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
```

3. Simpan file 04-refactor, kemudian jalankan file tersebut dari terminal dengan cara ketikkan node 04-refactor.js

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\04-refactor.js
Duplikat LAMBAT O(n^2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.11 Membuat Folder dan File Tugas

1. Didalam praktikum-5, buat folder baru Bernama tugas.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ mkdir tugas
```

3. Selanjutnya, buat dua file tugas didalam folder tugas.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma/praktikum-
5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

3.12 Membuat Tugas 1

1. Pada tugas ini dilakukan analisis dan refactoring terhadap tiga fungsi. Fungsi A digunakan untuk mencari irisan dua array dengan versi $O(n^2)$ dan versi lebih efisien $O(n)$ menggunakan Set. Fungsi B digunakan untuk mengelompokkan anagram dengan memanfaatkan proses sorting sebagai key. Fungsi C digunakan untuk mengecek apakah terdapat dua angka yang jumlahnya sama dengan kuadrat angka lain, dengan versi $O(n^3)$ dan versi lebih efisien $O(n \log n)$.
   Setiap fungsi diberi analisis Big O (time dan space) serta dilakukan pengujian menggunakan data besar untuk membandingkan waktu eksekusi. Hasilnya menunjukkan bahwa versi yang telah direfactor memiliki performa yang lebih baik.

```
JS tugas-1.js ×   JS tugas-2.js
tugas > JS tugas-1.js > ...
//Fungsi A: Intersection
// O(n^2)
function intersectionLambat(a, b) {
    let hasil = [];
    for (let i = 0; i < a.length; i++) {
        for (let j = 0; j < b.length; j++) {
            if (a[i] === b[j]) hasil.push(a[i]);
        }
    }
    return hasil;
}

// O(n)
function intersectionCepat(a, b) {
    let set = new Set(b);
    return a.filter(x => set.has(x));
}

// Fungsi B: Anagaram
function kelompokAnagram(arr) {
    let map = {};
    for (let kata of arr) {
        let key = kata.split('').sort().join('');
        if (!map[key]) map[key] = [];
        map[key].push(kata);
    }
    return Object.values(map);
}

// Fungsi C: Cek Kuadrat
// O(n^3)
function cekKuadratLambat(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = 0; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (arr[i] + arr[j] === arr[k] * arr[k]) return true;
            }
        }
    }
    return false;
}

// O(n log n)
function cekKuadratCepat(arr) {
    let set = new Set(arr);
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            let jumlah = arr[i] + arr[j];
            let akar = Math.sqrt(jumlah);
            if (Number.isInteger(akar) && set.has(akar)) return true;
        }
    }
    return false;
}

// TEST
console.log('Fungsi A:');
console.log(intersectionLambat([1,2,3,4], [3,4,5]));
console.log(intersectionCepat([1,2,3,4], [3,4,5]));

console.log('\n Fungsi B:');
console.log(kelompokAnagram(['eat', 'tea', 'tan', 'ate', 'nat', 'bat']));

console.log('\n Fungsi C:');
console.log(cekKuadratLambat([1,2,3,4,5]));
console.log(cekKuadratCepat([1,2,3,4,5]));
```

2. Simpan dan jalankan file tersebut dari terminal dengan cara ketikkan node tugas/tugas-1.js

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\tugas\tugas-1.js
Fungsi A:
[ 3, 4 ]
[ 3, 4 ]

 Fungsi B:
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

 Fungsi C:
true
true
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.13 Membuat Tugas 2

1. Pada tugas ini dilakukan pengujian performa beberapa fungsi dengan kompleksitas berbeda, yaitu O(1)**,** O(n)**,** O(n log n)**,** dan O(n^2). Setiap fungsi dijalankan menggunakan berbagai ukuran data untuk mengukur waktu eksekusi. Pengujian dilakukan menggunakan fungsi benchmarkSemua dengan nilai input [100, 5000, 1000, 5000, 10000]. Waktu eksekusi diukur menggunakan Date.now() untuk melihat perbedaan peforma dari masing masing kompleksitas. Hasil pengamatan menunjukkan bahwa waktu eksekusi meningkat seiring dengan bertambahnya kompleksitas algoritma. Pola pertumbuhan yang diperoleh sesuai dengan teori Big O, di mana algoritma dengan kompleksitas lebih tinggi membutuhkan waktu yang lebih lama.

```javascript
// O(1)
function fn_O1(n) {
    return n + 1;
}

// O(n)
function fn_On(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        total += i;
    }
    return total;
}

// O(n log n)
function fn_OnLogn(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) { // log2(n)
            total += j;
        }
    }
    return total;
}

// O(n^2)
function fn_On2(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            total += i + j;
        }
    }
    return total;
}

// Fungsi ukur waktu
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(label + ":", selesai - mulai, "ms");
}

// Benchmark
function benchmarkSemua(ukuranData) {
    for (let n of ukuranData) {
        console.log("\nUkuran data:", n);

        ukurWaktu("O(1)     ", () => fn_O1(n));
        ukurWaktu("O(n)     ", () => fn_On(n));
        ukurWaktu("O(n log n)", () => fn_OnLogn(n));
        ukurWaktu("O(n^2)   ", () => fn_On2(n));
    }
}

// panggil
benchmarkSemua([100, 500, 1000, 5000, 10000]);

// ANALISIS
/*
Hasil pengamatan:
- O(1) hampir tidak berubah walaupun n bertambah
- O(n) meningkat secara linear
- O(n log n) lebih cepat dari O(n^2) tetapi lebih lambat dari O(n)
- O(n^2) meningkat sangat cepat saat n besar
```

2. Simpan dan jalankan diterminal.

```
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5> node .\tugas\tugas-2.js

Ukuran data: 100
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 0 ms

Ukuran data: 500
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 1 ms

Ukuran data: 1000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 2 ms

Ukuran data: 5000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 33 ms

Ukuran data: 10000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 112 ms
PS D:\SEMESTER_GENAP_2026\2025573010006_strukturdatadanalgoritma\praktikum-5>
```

3.14 Mengumpulkan Tugas di Repository

1. Membuka Git Bash dan masuk ke folder repository.

```
MINGW64:/d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma

ACER@LAPTOP-ENDOAFFH MINGW64 ~ (master)
$ cd /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma

ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatada
nalgoritma (main)
```

2. Menambah folder praktikum-5/ ke staging.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time
 Git touches it
```

3. Mengecek status file.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   praktikum-5/01-intro-kompleksitas.js
        new file:   praktikum-5/02-kelas-big-o.js
        new file:   praktikum-5/03-space-complexity.js
        new file:   praktikum-5/04-refactor.js
        new file:   praktikum-5/latihan-1.js
        new file:   praktikum-5/package.json
        new file:   praktikum-5/tugas/tugas-1.js
        new file:   praktikum-5/tugas/tugas-2.js
```

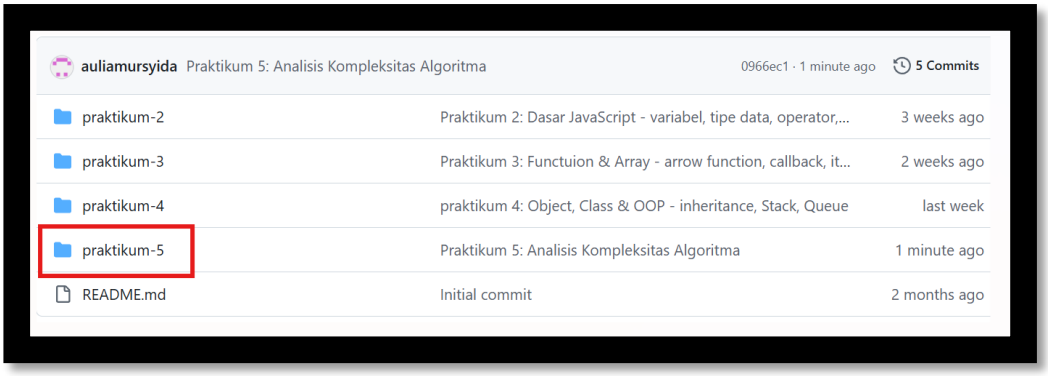
4. Melakukan commit

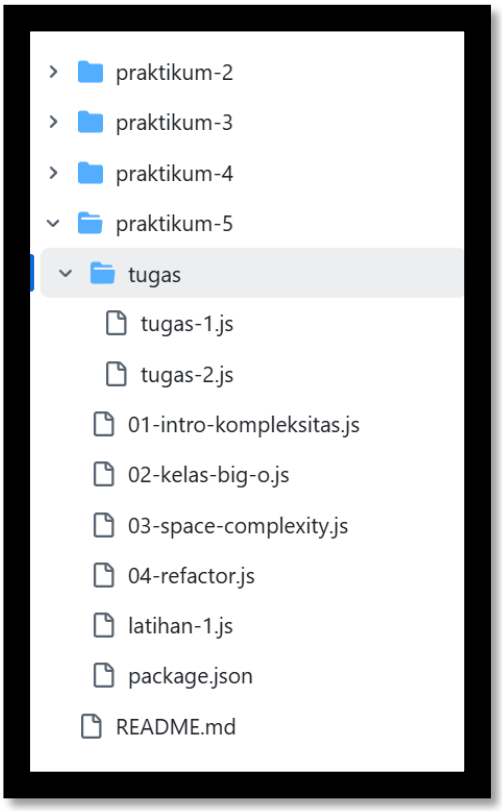
```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 0966ec1] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 488 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

5. Mengirim file ke repository.

```
ACER@LAPTOP-ENDOAFFH MINGW64 /d/SEMESTER_GENAP_2026/2025573010006_strukturdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.00 KiB | 1.67 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/auliamursyida/2025573010006_strukturdatadanalgoritma.git
   f9c16c1..0966ec1  main -> main
```

6. Selanjutnya buka akun GitHub di browser pencarian, cek apakah file sudah berhasil dikirim.





BAB IV
ANALISIS

4.1 Analisis Efisiensi Algoritma

Berdasarkan praktikum yang dilakukan, efisiensi algoritma dapat dilihat dari waktu eksekusi dan penggunaan memori. Algoritma yang efisien mampu menyelesaikan masalah dengan lebih cepat dan menggunakan sumber daya yang lebih sedikit. Dari hasil percobaan, terlihat bahwa algoritma dengan kompleksitas kecil bekerja lebih cepat dibandingkan algoritma dengan kompleksitas besar, terutama ketika jumlah data meningkat. Hal ini menunjukkan bahwa pemilihan algoritma sangat berpengaruh terhadap performa program.

4.2 Analisi notasi big O

Notasi Big O digunakan untuk menggambarkan pertumbuhan kinerja algoritma terhadap ukuran input. Berdasarkan hasil pengujian:

1. Algoritma O(1) memiliki waktu eksekusi paling cepat karena tidak bergantung pada jumlah data.
2. Algoritma O(n) membutuhkan waktu yang bertambah seiring jumlah data.
3. Algoritma $O(n^2)$ jauh lebih lambat karena menggunakan nested loop.
4. Algoritma O(log n) lebih efisien dibandingkan O(n) karena mengurangi data setiap langkah.

Dari analisis tersebut, dapat disimpulkan bahwa semakin kecil kompleksitas Big O, maka semakin baik performa algoritma.

4.3 Analisis Space Complexity

Space complexity menunjukkan jumlah memori tambahan yang digunakan oleh algoritma. Dari praktikum:

1. Algoritma dengan O(1) hanya menggunakan sedikit variabel tambahan.

2. Algoritma dengan O(n) menggunakan memori tambahan sesuai jumlah data, seperti pembuatan array baru.
3. Algoritma rekursif juga dapat menggunakan memori besar karena adanya call stack.

Hal ini menunjukkan adanya hubungan antara penggunaan memori dan efisiensi algoritma.

4.4 Analisis Mengenali Pola Kompleksitas

Dalam praktikum, pola kompleksitas dapat dikenali dari struktur kode, seperti:

1. Akses langsung ke data : O(1)
2. Perulangan tunggal : O(n)
3. Perulangan bersarang (nested loop) : $O(n^2)$
4. Perulangan dengan pembagian : O(log n)

4.5 Stack dan Queue

Implementasi stack dan queue menunjukkan cara pengelolaan data dengan aturan tertentu. Stack menggunakan prinsip LIFO (data terakhir masuk keluar pertama), sedangkan queue menggunakan FIFO (data pertama masuk keluar pertama). Kedua struktur ini sangat berguna dalam berbagai kasus seperti pengelolaan antrian dan proses data.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa efisiensi algoritma sangat penting dalam pemrograman, terutama ketika menangani data dalam jumlah besar. Penggunaan Notasi Big O membantu dalam memahami kinerja algoritma dari segi waktu dan memori. Algoritma dengan kompleksitas kecil seperti O(1) dan O(log n) memiliki performa yang lebih baik dibandingkan algoritma dengan kompleksitas besar seperti $O(n^2)$ dan $O(2^n)$. Selain itu, penggunaan struktur data yang tepat dapat meningkatkan efisiensi algoritma secara signifikan.